TÀI LIỆU KỸ THUẬT · BẢN TỔNG HỢP TOÀN HỆ THỐNG

GoHub Intel

Quy trình, mô hình và sơ đồ của toàn bộ hệ thống: dữ liệu, analytics, trợ lý AI, tích hợp Lark, tác vụ định kỳ, bảo mật, vận hành và kế hoạch mở rộng. Viết để người mới đọc một lần là hiểu hệ thống chạy thế nào, chỗ nào chậm, chỗ nào rủi ro.

THÔNG TIN	GIÁ TRỊ
Phiên bản tài liệu	2026-10-28 (s243)
Nhánh / commit	`main` = `e4cee3d4`, `staging` bằng `main` tại thời điểm viết
Môi trường	production `intel-v2.gohub.cloud` · staging `stg-intel-v2.gohub.cloud`
Nguồn	mã nguồn, `docs/wiki`, `CLAUDE.md`, số đo trực tiếp trên `gohub_dw` (chỉ SELECT)
Cách đọc	Mỗi chương độc lập. Nhãn `đã kiểm` là đã đối chiếu mã/số đo; `theo wiki` là lấy từ tài liệu, chưa đọc lại mã hôm nay; `cần xác nhận` là chưa chắc.

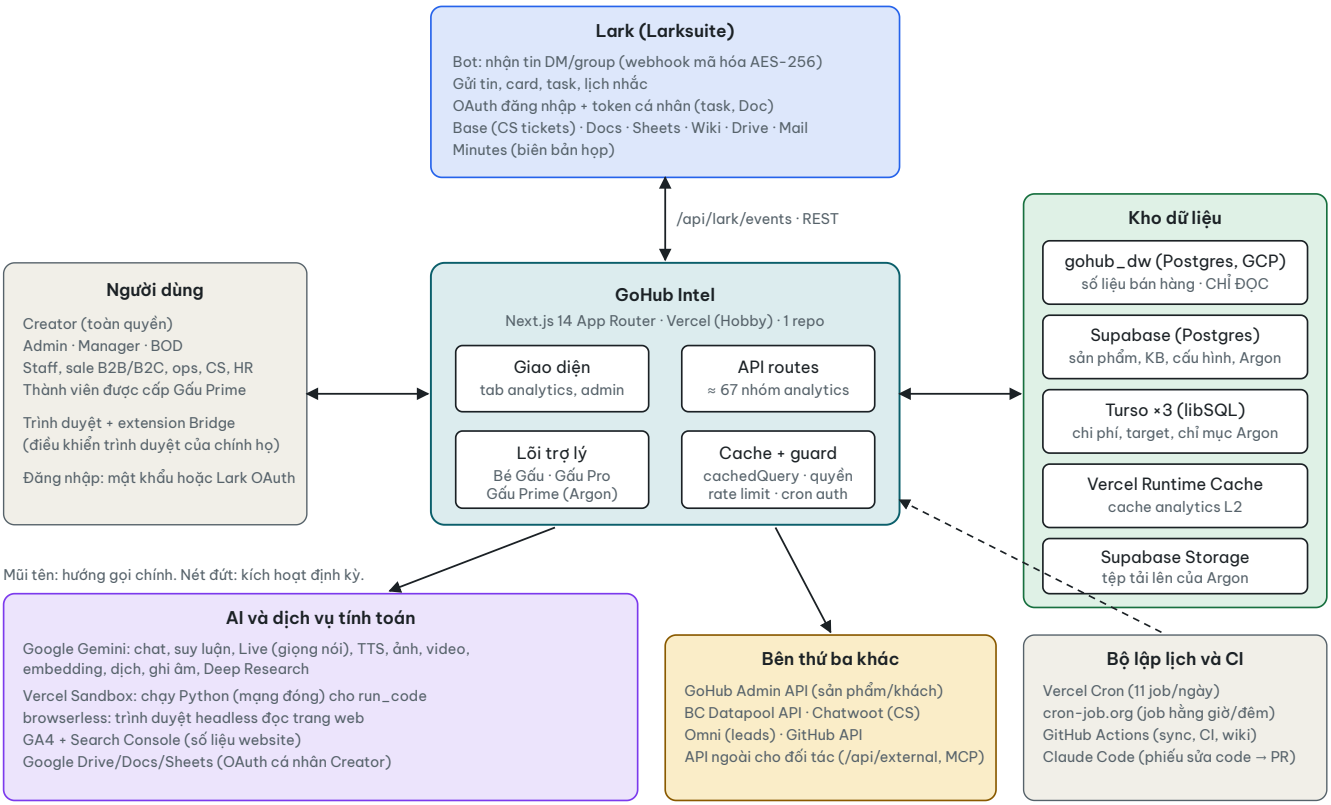
Tài liệu bổ trợ: kế hoạch Rollup Analytics (`docs/plans/analytics-rollup.md` và bản PDF cùng thư mục), wiki từng tab (`docs/wiki/system/tabs/`), nhật ký `docs/session_summary.txt`.

Mục lục

1 Bức tranh tổng thể
2 Hạ tầng, môi trường, phát hành
3 Các kho dữ liệu
4 Luồng nạp dữ liệu
5 Analytics: vòng đời một request và cache
6 Quy tắc nghiệp vụ tính số
7 Danh mục tab và module
8 Xác thực và phân quyền
9 Hệ trợ lý AI
10 Tích hợp Lark
11 Bridge: extension trình duyệt
12 Tác vụ định kỳ
13 Bảo mật và an toàn
14 Hiệu năng, giới hạn, Rollup
15 Vận hành và xử lý sự cố
A Phụ lục: bảng dữ liệu, biến môi trường, thuật ngữ, nợ kỹ thuật

1. Bức tranh tổng thể

GoHub Intel là web nội bộ (Next.js 14) gom ba việc: **báo cáo số liệu kinh doanh** (analytics), **quản lý sản phẩm và nhà cung cấp**, và **trợ lý AI** (Bé Gấu, Gấu Pro, Gấu Prime) tích hợp sâu với Lark. Toàn bộ chạy serverless trên Vercel; dữ liệu nằm ở nhiều kho tách vai trò.



Cách đọc: một ứng dụng duy nhất làm trung tâm. Người dùng và Lark gọi vào; ứng dụng đọc ghi 5 loại kho, gọi AI và dịch vụ ngoài; các bộ lập lịch gọi ngược vào các route `/api/cron/*`.

Ba trụ của hệ thống

1. Analytics

≈ 24 tab báo cáo (Dashboard, Quarter, BOD, Channels, B2B, B2C, Staff, Customers, Vendors, Orders, Products…). Số liệu đọc từ `gohub_dw`, cộng chi phí/target từ Supabase và Turso, tính CM1 trong code.

2. Sản phẩm và NCC

Catalog 4 tầng (Product, SKU, Listing, Item) đồng bộ từ API GoHub; danh mục nhà cung cấp, bảng giá, tính giá, tạo sản phẩm, khuyến mãi, BC Datapool, 3HK Data Usage.

3. Trợ lý AI

Bé Gấu (chatbot toàn team), Gấu Pro (trợ lý agent cho Creator), Gấu Prime (Argon: tham mưu theo Sứ mệnh, Kho tri thức, mở cho thành viên được cấp).

Vai trò người dùng

VAI TRÒ	THẤY GÌ	GHI CHÚ
`creator`	Mọi tab, mọi API, nhóm "Creator" (Gấu Pro, Gấu Prime, Dữ liệu và API, Bridge, Creator Settings…)	Chủ hệ thống, một vài người

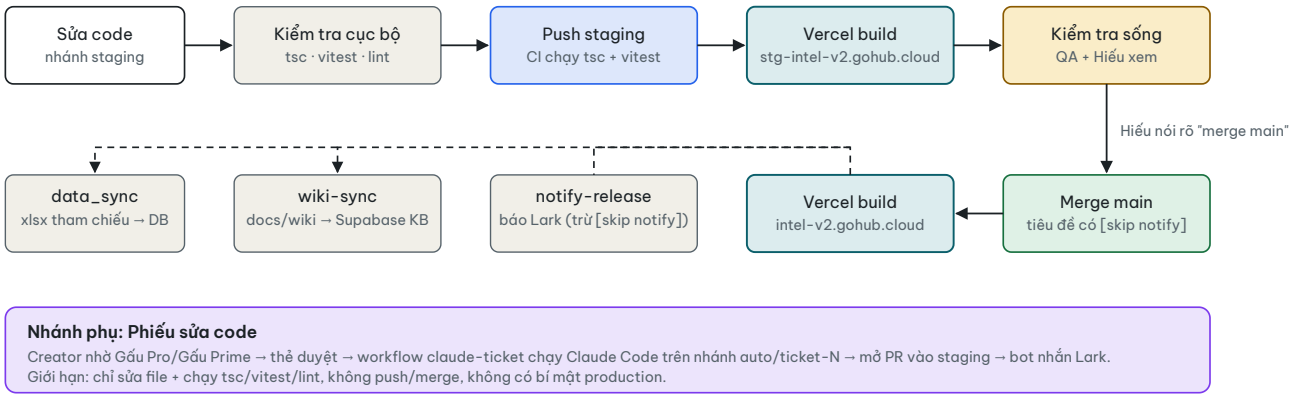
`admin`, `manager`	Hầu hết analytics + quản trị (Người dùng và quyền, Cấu hình, Dữ liệu và giá)	Admin toàn quyền analytics; manager trừ công cụ admin
`bod` và các vai trò vận hành	Theo ma trận `role_permissions` trong `app_settings` (deny-by-default)	staff, b2b, b2c, saleb2c, ops-&-cs, hr, product…
Thành viên Gấu Prime	Chỉ mục Hỏi của Gấu Prime với quyền được Creator cấp	Danh sách `argon_users`; hạn mức 40 lượt và 3 USD/ngày

2. Hạ tầng, môi trường và quy trình phát hành

Một repo GitHub, hai nhánh chính, hai domain Vercel. Điểm cần nhớ nhất: **staging và production dùng CHUNG cơ sở dữ liệu**; chỉ khác code và cache.

Môi trường

MỤC	PRODUCTION	STAGING
Domain	`intel-v2.gohub.cloud`	`stg-intel-v2.gohub.cloud`
Nhánh Git	`main`	`staging`
Cơ sở dữ liệu	cùng `gohub_dw`, cùng Supabase, cùng Turso (dữ liệu thử trên staging là dữ liệu thật: cuộc thử tạo bằng API phải xoá)	
Runtime Cache	riêng (namespace `production`)	riêng (`preview`)
Cron Vercel	có (vercel.json, chỉ production)	không; cron ngoài đã trỏ production
Biến môi trường	`ANALYTICS_DB_*` phải tick scope Preview, thiếu thì truy vấn trả rỗng âm thầm	



Quy tắc cứng: mọi thay đổi vào `staging` trước; KHÔNG tự merge `main` khi chưa được Hiếu yêu cầu rõ. Migration SQL chạy **sau** merge (migration kiểu DROP đã từng gây sự cố khi chạy trước).

GitHub Actions

WORKFLOW	KÍCH HOẠT	VIỆC
`ci`	push/PR vào staging, main	tsc + vitest (timeout 10')
`sync`	01:00 UTC hằng ngày (job `core`), 02:00 UTC Chủ nhật (job `items`), chạy tay	kéo products/skus/listings (hằng ngày) và items 233k dòng (hằng tuần, ~100') từ API GoHub về Supabase; chạy theo `main`
`data_sync`	push `main` có file xlsx trong `resources/reference/`	nhập file tham chiếu vào Supabase (kiểm băm)
`wiki-sync`	push `main` có `docs/wiki/**`	đồng bộ wiki vào `kb_wiki_pages` (Chatbot tra cứu)
`notify-release`	push `main`	báo Lark danh sách thay đổi + backlog còn trên staging (bỏ qua nếu commit cuối có `[skip notify]`)

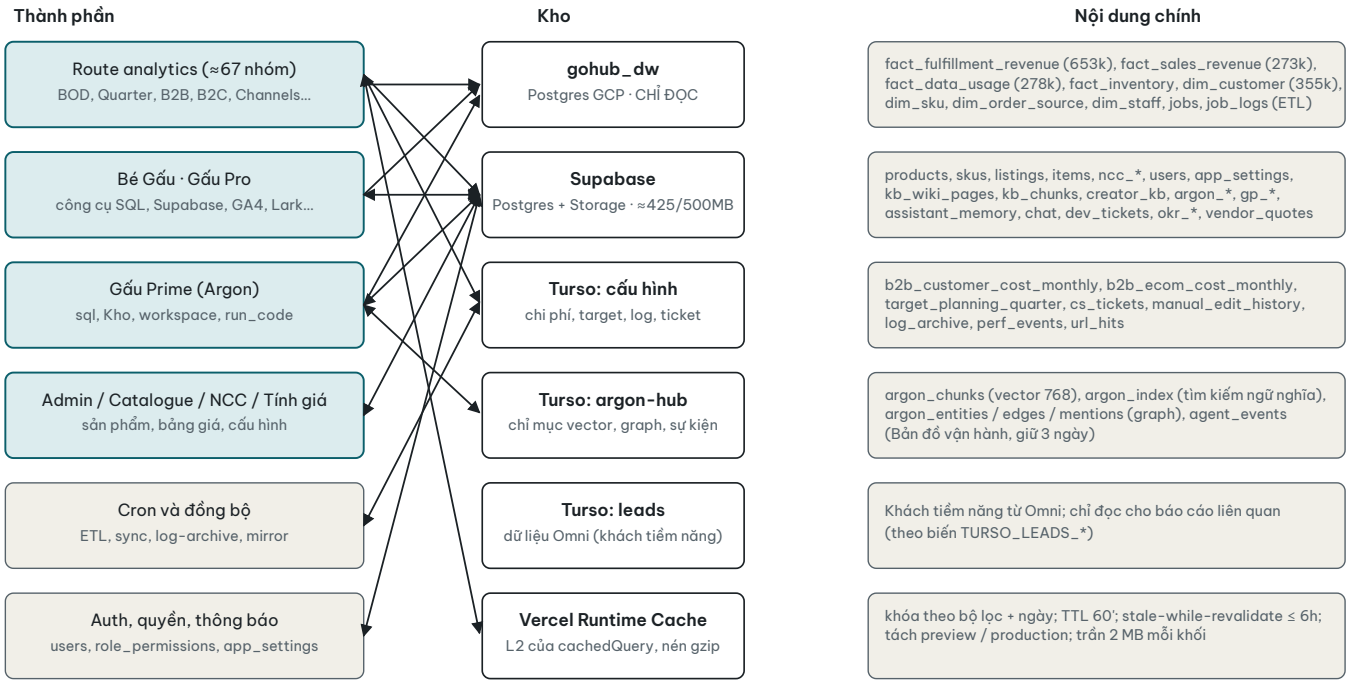
`claude-ticket`	`workflow_dispatch` từ web sau khi Creator duyệt phiếu	Claude Code sửa trên nhánh riêng, mở PR vào staging
`broadcast-once`, `scheduled-messages`	chạy tay	phát thông báo một lần; kích hoạt tay gửi tin lịch (bình thường do cron-job.org gọi)

Giới hạn của gói miễn phí đang dùng

DỊCH VỤ	GIỚI HẠN LIÊN QUAN	HỆ QUẢ
Vercel Hobby	cron tối đa 1 lần/ngày/job; thời gian hàm có trần	job hằng giờ phải do cron-job.org gọi; nhiều route đặt `maxDuration` 60–300s
Supabase (free)	500 MB; hiện ≈ 425 MB	không thêm dữ liệu lớn; đã dọn bảng/index, chuyển log và ticket sang Turso
Turso (free)	5 GB, 500 triệu lượt đọc, 10 triệu lượt ghi mỗi tháng (theo trang giá)	nơi chứa dữ liệu mới nếu cần; phải theo dõi lượt ghi
GitHub	repo công khai (theo GitHub API)	tuyệt đối không commit bí mật; `CLAUDE.md`, `Hieu/`, `.env*` nằm ngoài git

3. Các kho dữ liệu

Mỗi kho có một vai trò. Quy tắc chung: `gohub_dw` là nguồn sự thật cho số liệu bán hàng và **không bao giờ bị ghi**; mọi thứ do hệ thống tự tạo (cấu hình, tri thức, trí nhớ, chỉ mục) nằm ở Supabase hoặc Turso.

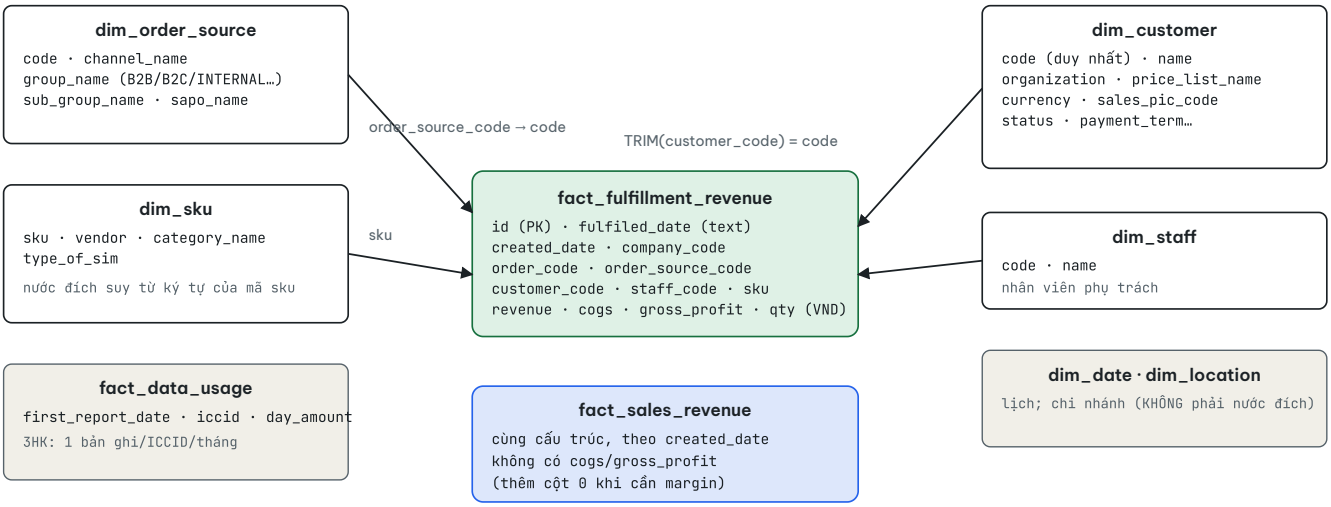


Đọc: hai chiều mũi tên là đọc và ghi. Chỉ có đường vào `gohub_dw` là một chiều (đọc). Gấu Prime và Gấu Pro đọc dữ liệu công ty qua các công cụ chỉ đọc; chỉ mục tìm kiếm và graph của Argon nằm riêng ở `argon-hub` để không phình Supabase.

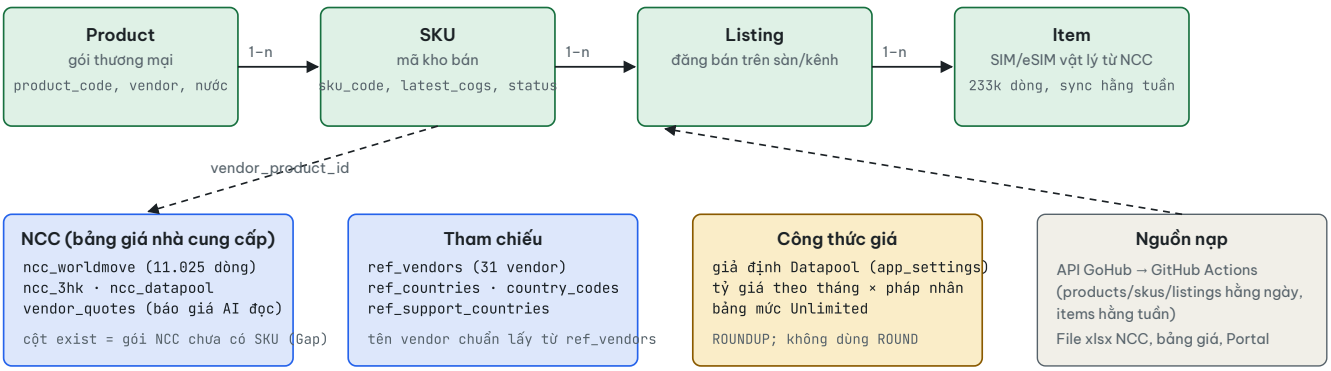
gohub_dw: kho số liệu bán hàng `đã kiểm`

BẢNG	DÒNG (28/10/2026)	DUNG LƯỢNG	VAI TRÒ
`fact_fulfillment_revenue`	652.741	186 MB	doanh thu theo ngày **giao** (mặc định của hầu hết báo cáo); có giá vốn, GP
`fact_sales_revenue`	272.889	105 MB	doanh thu theo ngày **tạo đơn**; không có giá vốn (margin = 0)
`fact_data_usage`	278.053	62 MB	tiêu hao data 3HK, 1 bản ghi/ICCID/tháng
`dim_customer`	355.420	41 MB	khách hàng (`code` duy nhất): tên, tổ chức, price list, nhân viên phụ trách…
`dim_sku`	29.306	13 MB	SKU: vendor, loại SIM, danh mục
`dim_order_source`	138 (76 dùng trong fact)	128 kB	kênh bán: nhóm B2B/B2C, kênh con
`dim_staff` , `dim_date` , `dim_warehouse` , `fact_inventory`	155 / 1.462 / 13 / 3.251	nhỏ	nhân viên, lịch, kho, tồn kho
`jobs` , `job_logs`	—	—	lịch và lịch sử các ETL (dùng để biết khi nào dữ liệu mới về)

Điểm yếu cấu trúc: chỉ có khóa chính (`id`), không có chỉ mục theo ngày, kênh hay khách; cột ngày là `varchar` 10 ký tự. Mọi truy vấn quét cả bảng. Hiếu không có quyền DDL, nên giải pháp mở rộng phải nằm ngoài `gohub_dw` (xem chương 14).



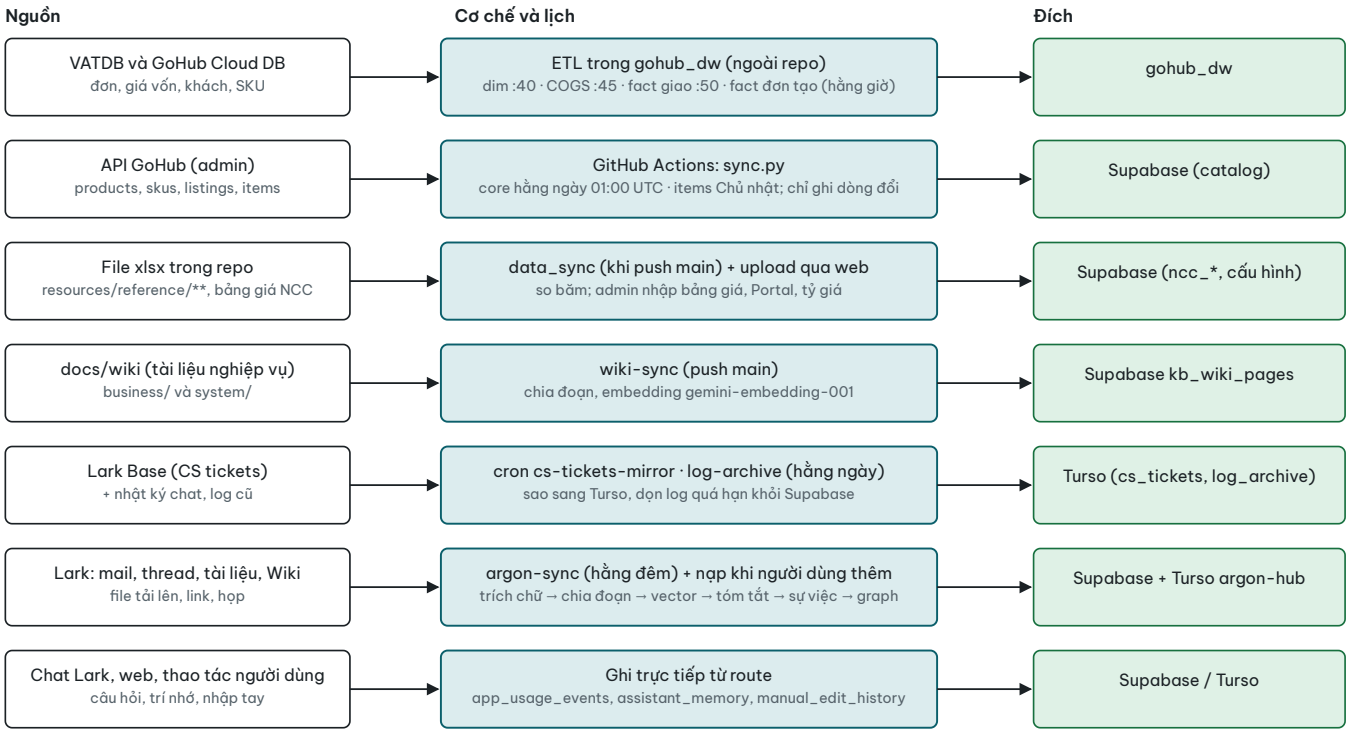
Lưu ý khi viết SQL: không bọc `TRIM()` lên cột phía `dim_customer` khi join (từ 1s lên ~10s). Luôn lọc ngày qua `getDateFilter` . Nước đích của SKU không lấy từ `dim_location` .



Catalog tách bốn tầng để một gói bán được qua nhiều kênh và nhiều nguồn SIM mà không trộn dữ liệu. NCC và bảng giá nằm ngoài chuỗi này, chỉ nối bằng `vendor_product_id` .

4. Luồng nạp dữ liệu

Dữ liệu vào hệ thống qua bảy đường. Chỉ đường 1 (ETL của kho số liệu) nằm ngoài mã nguồn này; các đường còn lại do repo điều khiển.



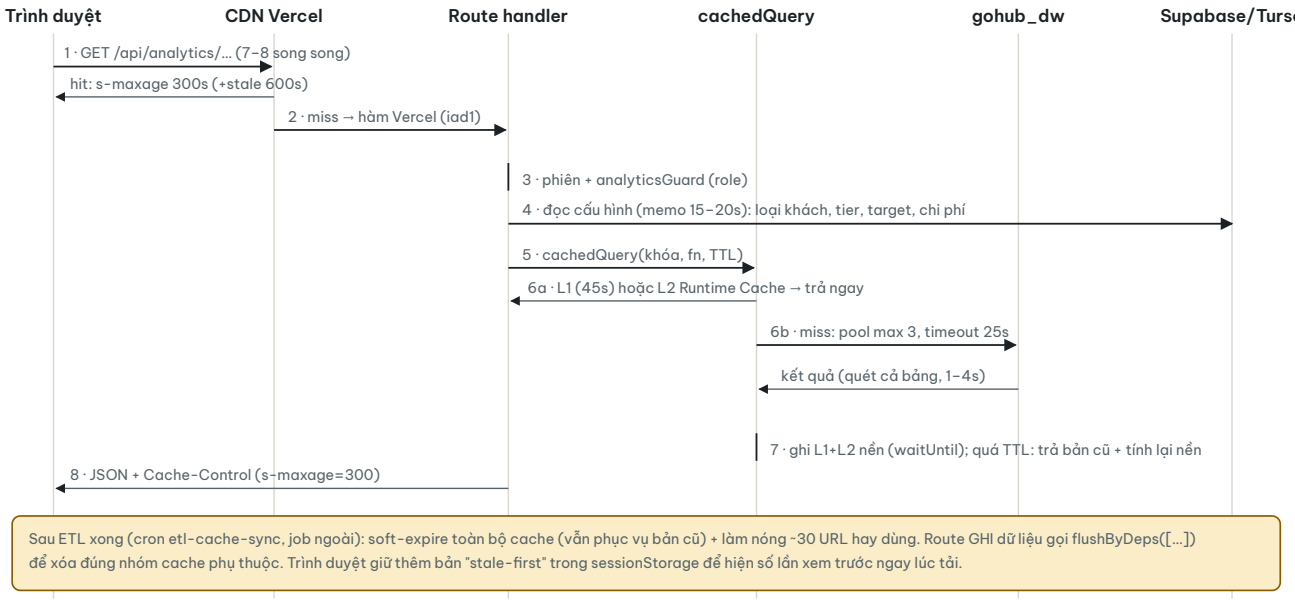
Đường 1 quyết định độ tươi của mọi báo cáo: số liệu mới chỉ xuất hiện sau khi ETL xong, và app chỉ lấy dữ liệu đến `CURRENT_DATE - 1`. Lịch ETL đơn tạo hiện chưa xác nhận.

Ngân sách dữ liệu

- Supabase gần đầy (≈ 425/500 MB): nguyên nhân từng là chỉ mục của `items` / `listings`. Đã bỏ 10 bảng chết, bỏ index thừa, chỉ ghi dòng đổi khi sync, chuyển ticket CS và log sang Turso.
- Không thêm dữ liệu lớn vào Supabase; dữ liệu mới ưu tiên Turso.

5. Analytics: vòng đời một request và cache

Mỗi tab gọi nhiều route; mỗi route dựng SQL, chạy trên `gohub_dw`, gộp với cấu hình ở Supabase/Turso rồi trả về. Cache che các lần xem lặp lại.



Đọc: bước 6b là chỗ chậm. Với ~650k dòng thô, một truy vấn nặng tốn 1,5–3,7s lần đầu, và mỗi route thường chạy 2–10 truy vấn nên một tab lúc cache nguội mất 6–12s.

Bốn lớp cache `đã kiểm`

LỚP	NƠI	HẠN	LÀM MỚI / XOÁ
Trình duyệt	`sessionStorage` (`stale-first`)	trong phiên	luôn gọi lại và thay bằng số mới
CDN	`CACHE_HEADERS`: `s-maxage=300, stale-while-revalidate=600`	5 phút + 10 phút stale	theo URL; không xoá được chủ động
L1	bộ nhớ instance (`Map`)	45 giây	hết hạn; flush theo `deps`
L2	Vercel Runtime Cache (`@vercel/functions`), nén > 400 KB, trần 2 MB	TTL 60', stale ≤ 6h; quý đã đóng 24h/48h	`expireTag(dep:…)`, mốc epoch hard/soft

Quy tắc bắt buộc khi viết route analytics

- Khóa cache phải gồm mọi tham số ảnh hưởng kết quả, kể cả băm danh sách loại khách (`exclHash`); thiếu thì đổi cấu hình xong vẫn thấy số cũ.
- Gộp truy vấn cùng bộ lọc thành một (Quarter Report đã gộp 7 thành 2); I/O độc lập dùng `Promise.all`; tránh N+1.
- Không dùng `TRIM()` lên cột `dim_customer` trong JOIN; luôn lọc ngày bằng `getDateFilter` và kiểm tham số bằng `safeDate` / `safeCompanyCode`.
- Route mới dùng `analyticsGuard` thì tự vào danh sách làm nóng.
- Đo thật bằng `/api/analytics/perf-probe` (creator) và trang Data Map (`perf_events`) trước khi tối ưu.

6. Quy tắc nghiệp vụ tính số

Những định nghĩa dùng chung cho mọi tab. Khi hai tab lệch số, kiểm các quy tắc này trước.

Hai cơ sở thời gian

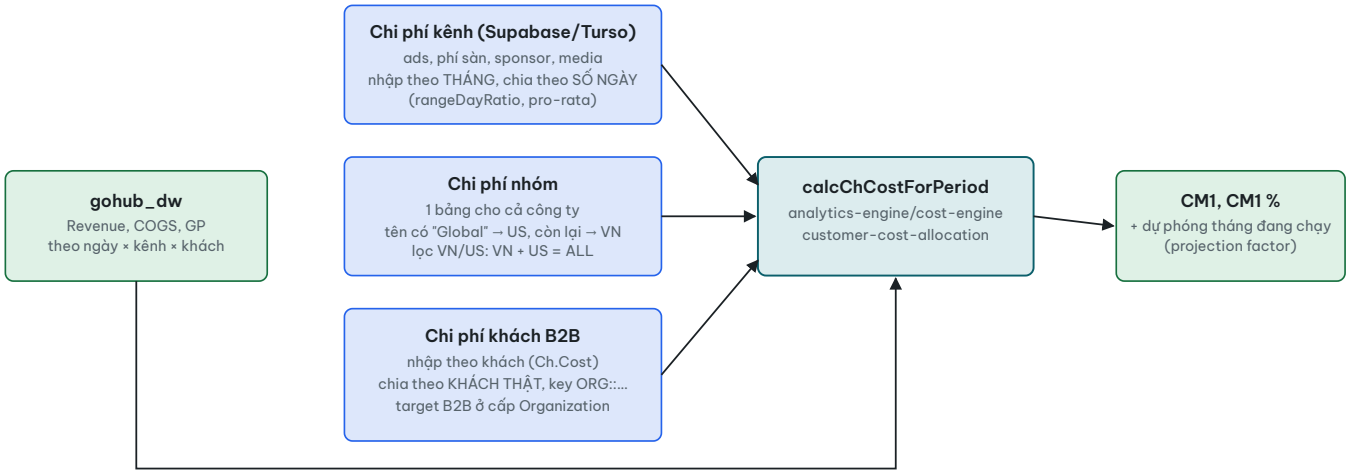
	FULFILLMENT (MẶC ĐỊNH)	CREATED (SALES)
Bảng	`fact_fulfillment_revenue`	`fact_sales_revenue`
Cột ngày	`fulfiled_date`	`created_date`
Doanh thu	`fulfilled_revenue_amount_vnd`	`sales_revenue_amount_vnd`
Giá vốn / GP	có	không có (= 0), nên mọi margin và CM1 bằng 0 là đúng, không phải lỗi

Bộ lọc chuẩn (mặc định tick = gồm hết)

BỘ LỌC	HIỆU ỨNG	MẶC ĐỊNH
Phí ship	loại SKU `SHIPPINGFEE0` khi bỏ tick	gồm
Đơn nội bộ	loại nhóm kênh `INTERNAL-TRANSACTION` (doanh thu 0, GP âm)	gồm
Khách Ops / danh sách loại trừ	loại theo MÃ hoặc TÊN khách trong cài đặt Quarter Report; B2C dùng danh sách riêng (`excludedForB2C`)	gồm
Khách INACTIVE	loại khách có `price_list_name` chứa INACTIVE	luôn loại
Công ty	`company_code` : ALL, VN, US	ALL

Chỉ số tài chính (chuẩn Management Report)

NHÃN	CÔNG THỨC	GHI CHÚ
Revenue	Σ doanh thu	
GP (Gross Profit)	Revenue − COGS	khóa dữ liệu `margin`
GPM %	GP / Revenue	
CM1	GP − chi phí vận hành (quảng cáo, phí sàn, tài trợ SP, media, chi phí khách, chi phí nhóm)	nhãn UI "CM1"; khóa dữ liệu cũ `gpm2` giữ nguyên
CM1 %	CM1 / Revenue	ngưỡng tiến độ: ≥100% xanh, ≥85% vàng, <85% đỏ
3HK contribution %	Revenue vendor 3HK Datapool / Revenue tổng	so tên vendor bằng `REPLACE(UPPER(vendor),' ','')='3HKDATAPOOL'`
%Datapool (KPI)	tỷ trọng doanh thu BC Datapool và 3HK	nhận cả hậu tố vendor `(CMHK)` , `(Singtel)`



CM1 không có trong `gohub_dw`: nó được tính trong code mỗi lần, từ số liệu bán hàng cộng ba nguồn chi phí. Vì vậy mỗi route phải gộp dữ liệu ở app, và đây cũng là lý do rollup chỉ lưu số cộng được (chương 14).

Phân loại khách và nhóm kênh

- **B2B / B2C:** theo `dim_order_source.group_name`. B2C luôn lọc `UPPER(group_name)='B2C'`.
- **Tier** (Strategic / VIP / Gold / Silver): một định nghĩa duy nhất qua `price_list_name` và `tierKeywords` của Quarter Report (`makeClassifyTier`). Bảng `partner_tiers` đã rỗng; mã cũ dựa vào nó đã thay thế nhưng còn vài chỗ thừa.
- **Nước đích của SKU:** suy từ vị trí ký tự mã SKU theo độ dài (13 ký tự: ký tự 3–5; 14: 1–3; 15: 2–4) và tra `country_codes` (Turso).

Quarter Report: mô hình mục tiêu

THÀNH PHẦN	NƠI LƯU	GHI CHÚ
Target tháng công ty (ALL/B2B/B2C)	`company_monthly_targets`	B2C = ALL − B2B
Target Squad 1/2	`squad_targets`	theo tháng
Target B2B theo Organization	`b2b_customer_targets` (khóa `ORG::<orgKey>`)	chia xuống mã con theo doanh thu
Target quý B2B/B2C	Turso	độc lập, chưa đồng bộ với target tháng
Cài đặt (tier, loại trừ khách)	`quarterly-settings`	đổi cấu hình tự làm mới cache qua băm trong khóa

7. Danh mục tab và module

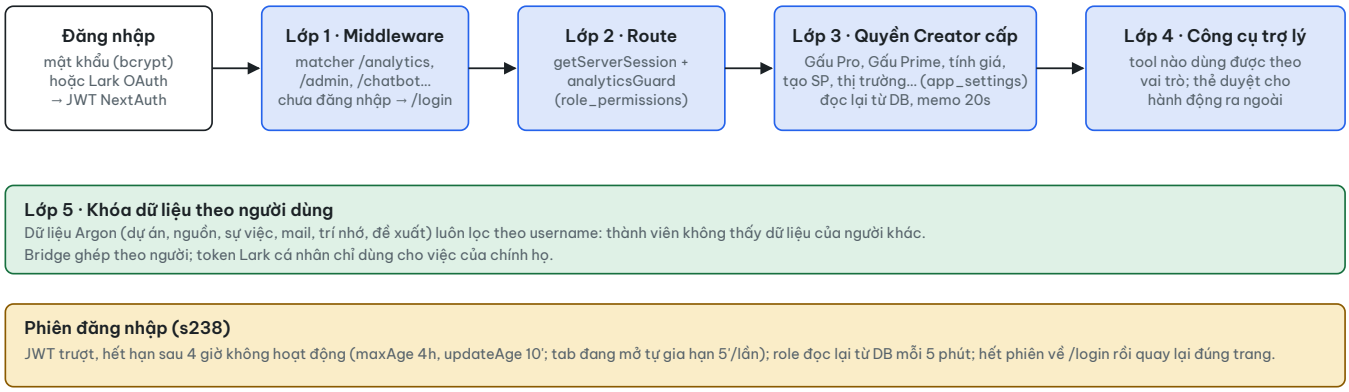
Điều hướng chia nhóm theo `lib/nav.ts` (và `components/sidebar.tsx`, hai nơi phải giữ đồng bộ). Cột "Quyền" là mặc định; Creator thấy tất cả.

NHÓM	TAB	MỤC ĐÍCH	DỮ LIỆU CHÍNH	
Overview	Dashboard	tổng quan, biểu đồ xu hướng	gohub_dw	
	Quarter Report	CM1 theo quý, Squad Progress, Performance (ALL/B2B/B2C), target, xuất Excel	gohub_dw + Supabase/Turso (target, chi phí)	
	BOD Report	báo cáo quản trị: 3 kỳ (này/trước/cùng kỳ năm ngoái), kênh, tier B2B	gohub_dw + chi phí	
	All-Time Report	lịch sử dài hạn (mặc định 12 tháng)	gohub_dw	
Sales Performance	Channels	xếp hạng kênh, "bán nhiều hay lãi cao"	gohub_dw + chi phí	
	B2B	Performance theo tháng/khoảng ngày, theo tier, khách lớn, VN Ecom	như trên	
	B2C	Performance, Metric, Advanced (GA4 Category)	gohub_dw + GA4	
	Website Analytics	GA4 và Search Console	GA4/GSC API	
	Staff, Customers, Vendors	hiệu suất nhân viên (target), khách B2B (lọc nhiều tầng), nhà cung cấp	gohub_dw + dim	
	Operations & Support	Orders	đơn hàng, lọc nhiều tầng có số đơn	gohub_dw (cấp đơn)
		Inventory	tồn kho, kế hoạch nhập, PO	gohub_dw + Supabase
3HK Data Usage		tiêu hao data 3HK	fact_data_usage	
CS Troubleshoot		ticket CS, khách B2B đọc từ dim_customer	Turso cs_tickets (mirror Lark Base)	
Analytics & Planning	Products (BI)	hiệu suất sản phẩm, B2B theo tier	gohub_dw	
	Product Catalogue	catalog dựng lại (không rành kỹ thuật vẫn xem được)	Supabase	
	Manage Costs (KPI/Target)	target và chi phí B2C	Supabase/Turso	
	Scheduled Messages	lịch gửi báo cáo vào Lark	Supabase	

Quản trị	Người dùng và quyền	user, ma trận quyền, 5 quyền Creator	Supabase
	Cấu hình hệ thống	Dữ liệu, Chatbot, API bên ngoài	app_settings
	Lịch sử nhập liệu	khôi phục số cũ ở ~21 route nhập tay	Turso manual_edit_history
	Dữ liệu và giá (Admin)	SKU destination, tỷ giá, công thức Datapool, soát giá vốn, tạo sản phẩm	Supabase
Creator	Creator Settings	cấp quyền Gấu Pro, Gấu Prime, tính giá, tạo SP, thị trường; kết nối Lark/Google	app_settings
	Gấu Pro	trợ lý agent đầy đủ	nhiều kho
	Gấu Prime (Argon)	Hôm nay, Hỏi, Kho, Thêm (Cuộc gọi, Bridge, Chạy song song, Tự động, Sứ mệnh cũ)	Supabase + Turso argon-hub
	Dữ liệu và API	SQL Query, Data Map, Giám sát dữ liệu (độ tươi, bất thường, đối chiếu), API Tester	gohub_dw (chỉ đọc)
	Bridge	cài và ghép extension	Supabase browser_bridge_*
	Thị trường và Báo giá	so giá vốn theo thị trường/vendor; AI đọc báo giá	Supabase vendor_quotes + ncc_*
	Tính giá và Tạo sản phẩm	tính giá 4 bước cho sale; tạo SP BC/3HK	Supabase
	BC Datapool	nhập báo giá và file Portal	Supabase
Khác	Bé Gấu	chatbot toàn team	nhiều kho
	Tổ Gấu	group chat nội bộ + tài liệu, hỏi AI	Supabase (Realtime)
	My Metrics	OKR: Tasks via Bé Gấu, KPI %Datapool, Kế hoạch quý, Lark events	Supabase okr_*, app_usage_events
	Promotions	khuyến mãi gắn SKU, nguồn cho chatbot	Supabase
	System SKUs, NCC Catalog, Reference	danh mục sản phẩm, NCC (Gap), quốc gia	Supabase

8. Xác thực và phân quyền

Nguyên tắc: **từ chối mặc định**. Quyền được kiểm ở nhiều lớp, từ middleware tới từng công cụ của trợ lý.



Một request có thể bị chặn ở bất kỳ lớp nào. Lớp 3 là chỗ Creator điều chỉnh quyền từng người mà không cần deploy.

Ma trận quyền Creator cấp (5 quyền)

QUYỀN	CHO PHÉP	KHÓA LƯU
Gấu Pro	dùng trợ lý agent (danh sách người dùng thử theo từng tính năng cá nhân)	`gp_allowed_users`, `gp_personal_features`
Gấu Prime	mục Hỏi với hạn mức/ngày	`argon_users`, `argon_member_limits`
Tính giá	công cụ tính giá cho sale	`pricing_users`
Tạo sản phẩm	tạo SP BC/3HK (gồm tính giá)	`product_create_users`
Thị trường và Báo giá	xem so giá, báo giá vendor	`market_users`

Cổng ngoài

- **Cron:** `Authorization: Bearer $CRON_SECRET`; thiếu biến môi trường thì từ chối (không bỏ qua).
- **API ngoài / MCP:** khóa tạo trong tab "API bên ngoài" (`external_api_keys`); chỉ đọc dữ liệu cho phép.
- **Lark webhook:** giải mã AES-256-CBC, xác minh `LARK_VERIFICATION_TOKEN`, loại trùng theo `event_id`.
- **Rate limit chat:** 20/10 lượt/phút theo người dùng; code hỗ trợ Upstash Redis dùng chung giữa instance, nhưng chưa chắc đã cấu hình trên Vercel (nếu thiếu thì chạy trong bộ nhớ từng instance) `cần xác nhận`.

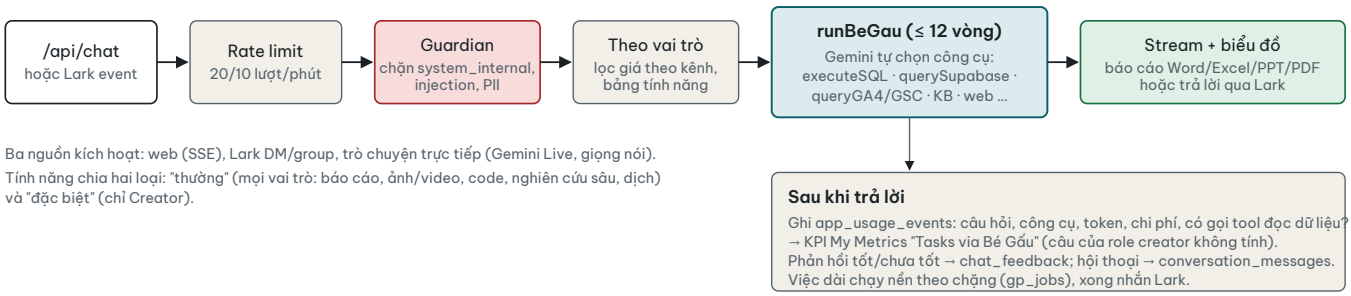
9. Hệ trợ lý AI

Ba trợ lý cùng dùng Gemini nhưng khác vai trò. Hướng đi đã chốt: dần chỉ còn hai (Bé Gấu và Gấu Prime); Gấu Pro giữ cho tới khi Gấu Prime đủ tiêu chí (trang "Chạy song song" theo dõi).

	Bé Gấu	Gấu Pro	Gấu Prime (Argon)
Người dùng	Mọi vai trò (web + Lark DM/group)	Creator; người được cấp từng tính năng cá nhân	Creator (đủ) + thành viên được cấp (quyền gần full, có hạn mức)
Vai trò	Chatbot tra cứu, trả lời nhanh	Làm theo lệnh: agent nhiều công cụ, việc nền, lịch	Tham mưu theo Sứ mệnh + Kho tri thức, trợ lý hợp nhất
Mã chính	`lib/agents/be-gau.ts`	`lib/agents/creator-ai.ts`	`lib/argon/*` (run-engine, tools…)
Vòng lặp	1 agent function-calling, ≤ 12 vòng; lõi chung agent-loop	≤ 20 vòng, SDK @google/genai, skills nạp khi cần; lõi chung	vòng riêng + vòng lõi chung (cờ argon_core_loop); router Nhanh/Đầy đủ
Công cụ	SQL (đọc), Supabase, sản phẩm, GA4/GSC, web, KB, báo cáo, tạo ảnh/video, code Python	20+: SQL, Supabase, Lark (task, Doc, Base, tin), Google, tệp máy, Bridge, portal, trí nhớ, lịch	sql, layDuLieuHeThong, Kho (sự việc, thực thể, mail), workspace, run_code, đọc trang, ảnh/video, Bridge, thẻ duyệt
An toàn	Guardian trước khi vào; lọc lộ tên bảng/cột trong câu trả lời	cổng duyệt hành động; chống prompt injection; nhật ký hành động	bậc quyền công cụ L0–L3; lượt đọc nội dung ngoài bị "nhiễm" và siết ghi
Trí nhớ	tra KB mỗi lượt; trí nhớ qua Lark DM; nhớ nhiễm theo cuộc chat	2 tầng (assistant_memory + theo hội thoại); học thói quen	dùng chung assistant_memory; chỉ ĐỀ XUẤT ghi nhớ, người dùng duyệt
Lưu	conversations, conversation_messages	gp_runs, gp_jobs, gp_pending_actions	argon_projects/missions/runs/sources…

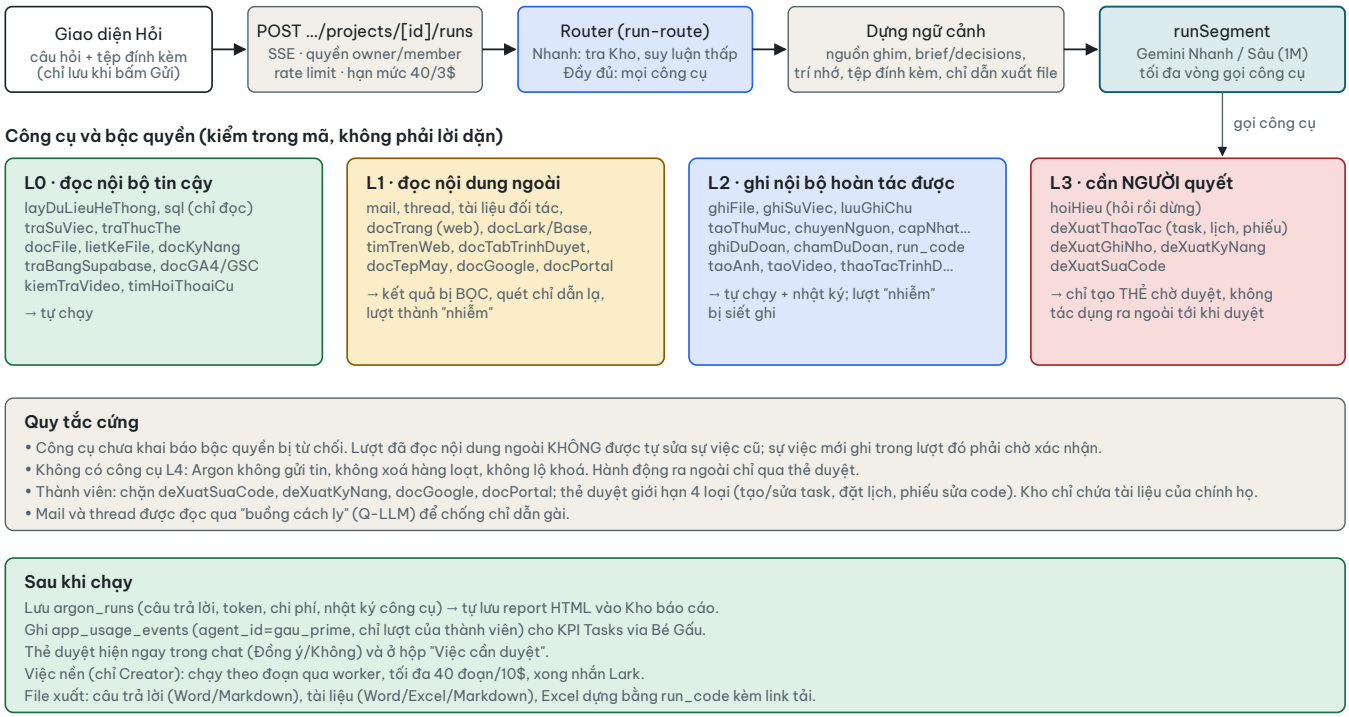
Bé Gấu và Gấu Pro chia nhau lõi agent (`lib/agents/core/agent-loop.ts`); Gấu Prime có kiến trúc riêng quanh Sứ mệnh và Kho nhưng dùng chung trí nhớ và thẻ duyệt với Gấu Pro.

9.1 Bé Gấu: một lượt trả lời



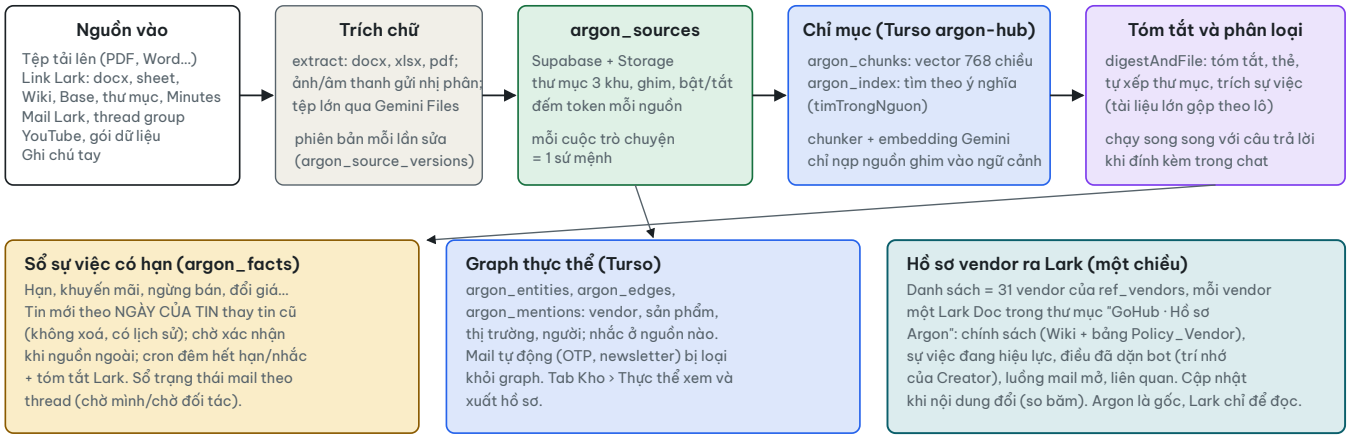
Nguồn: `docs/wiki/system/tabs/chatbot.md`, `analytics-creator-ai.md` `theo wiki`.

9.2 Gấu Prime (Argon): một lượt chạy



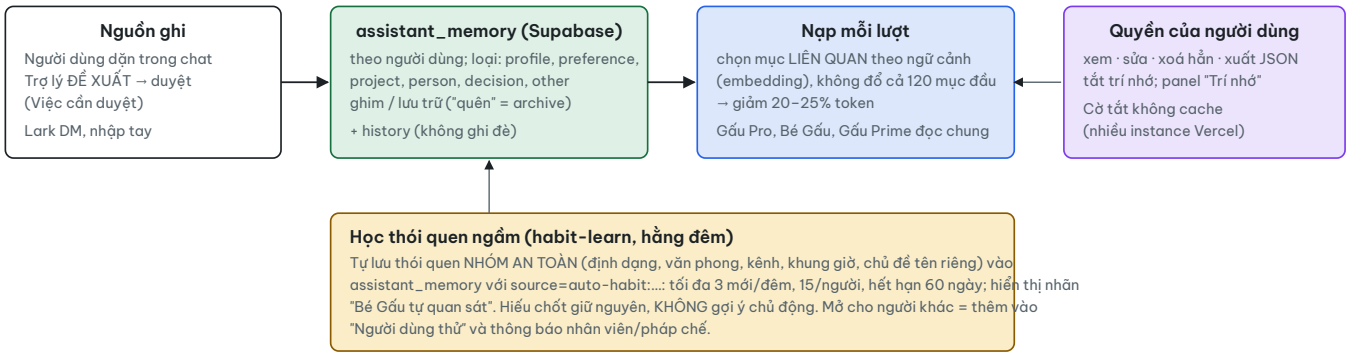
Nguồn: `lib/argon/run-safety.ts`, `run-engine.ts`, `access-pure.ts`, route `projects/[id]/runs` `đã kiểm`.

9.3 Kho tri thức của Gấu Prime



Công cụ đọc Kho: `traSuViec`, `traThucThe`, `traMail`, `timTrongNguon`, `docNguon`, `lietKeNguon`. Thành viên chỉ thấy Kho của chính mình; Creator có thêm tab Thực thể, xuất hồ sơ Lark và nguồn tự động.

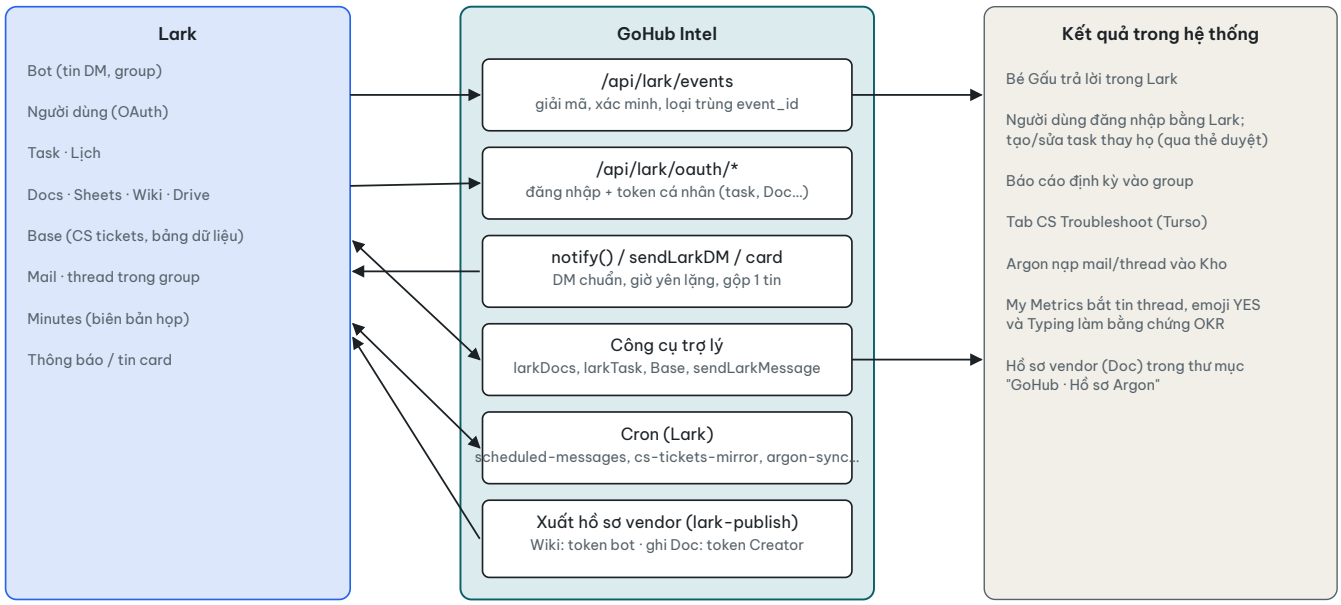
9.4 Trí nhớ



Hồ sơ vendor ra Lark chỉ dùng trí nhớ của Creator, loại `decision/project/person/other`, và chỉ các dòng nhắc đúng tên vendor.

10. Tích hợp Lark

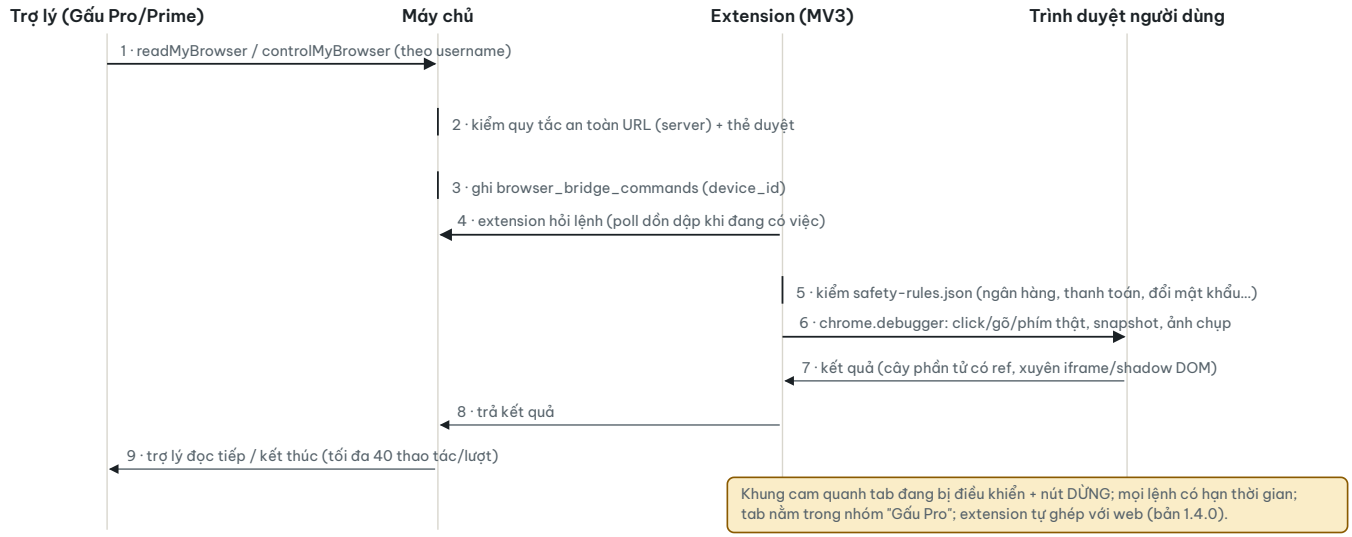
Lark là kênh làm việc chính của công ty: nhận câu hỏi, gửi báo cáo, lưu task, tài liệu và ticket. Ứng dụng dùng hai loại token: token của bot (tenant) và token OAuth cá nhân.



Token bot cần được thêm vào từng nơi muốn đọc: group chat, không gian Wiki, thư mục. Token cá nhân (OAuth) cần cấp đúng phạm vi (task, Doc...); thiếu quyền thì báo "Kết nối Lark lại" ở Creator Settings.

11. Bridge: extension trình duyệt

Cho trợ lý đọc và thao tác trên trình duyệt **của chính người dùng** (không phải trình duyệt headless của máy chủ). Mỗi người tự ghép trình duyệt của mình (đa người thuê).



Ghép thiết bị: `browser_bridge_pairings` / `devices`; ghi nhận thiết bị để truy vết. Điều kiện an toàn: lượt đã đọc nội dung ngoài (mail/web/Lark) thì KHÔNG được điều khiển trình duyệt (tránh bị điều khiển ngầm). Người dùng cũ phải tải bản mới và nạp lại extension khi có phiên bản mới.

12. Tác vụ định kỳ

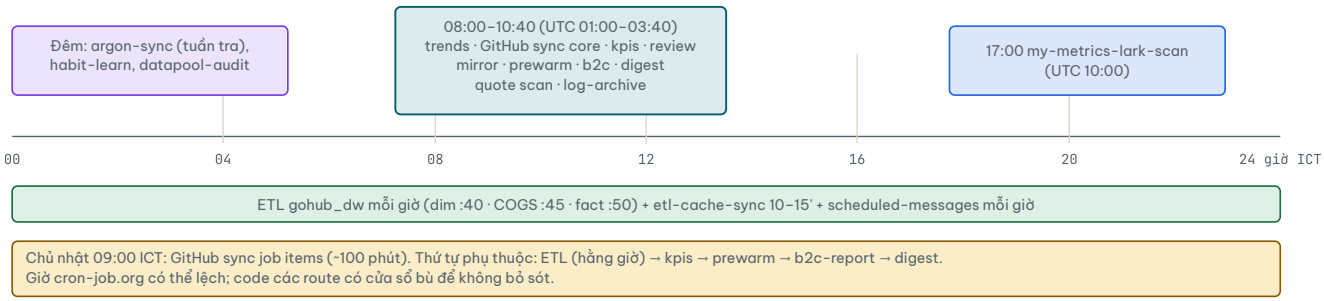
Ba nguồn kích hoạt: Vercel Cron (11 job, mỗi job tối đa 1 lần/ngày vì gói Hobby), cron-job.org (HTTP định kỳ, chính xác hơn, dùng cho job hằng giờ) và GitHub Actions. Mọi route cron kiểm `Bearer $CRON_SECRET`.

Vercel Cron (giờ UTC → ICT = UTC+7) đã kiểm

ROUTE	UTC	ICT	VIỆC
`scheduled-messages`	01:00	08:00	gửi tin Lark theo lịch (backstop; chính do cron-job.org gọi mỗi giờ, có cơ chế giành slot chống gửi trùng)
`refresh-trends`	01:00	08:00	thu thập trend (tìm web) vào `trend_snapshots`
`refresh-monthly-kpis`	01:30	08:30	tính `analytics_monthly_kpis`
`my-metrics-daily-review`	01:30	08:30	"Đánh giá hôm nay" + DM Lark 8:30
`cs-tickets-mirror`	02:00	09:00	kéo ticket CS từ Lark Base sang Turso
`prewarm-analytics`	02:00	09:00	làm nóng cache các URL/SQL hay dùng
`refresh-b2c-report`	02:30	09:30	snapshot B2C hằng tháng
`gau-pro-digest`	02:45	09:45	digest chủ động buổi sáng của Gấu Pro
`vendor-quote-scan`	03:15	10:15	quét báo giá vendor định kỳ
`log-archive`	03:40	10:40	sao log quá hạn sang Turso, dọn Supabase
`my-metrics-lark-scan`	10:00	17:00	quét tin nhắn Lark làm bằng chứng OKR

Job do cron-job.org gọi (cấu hình ngoài repo) theo wiki / CLAUDE.md

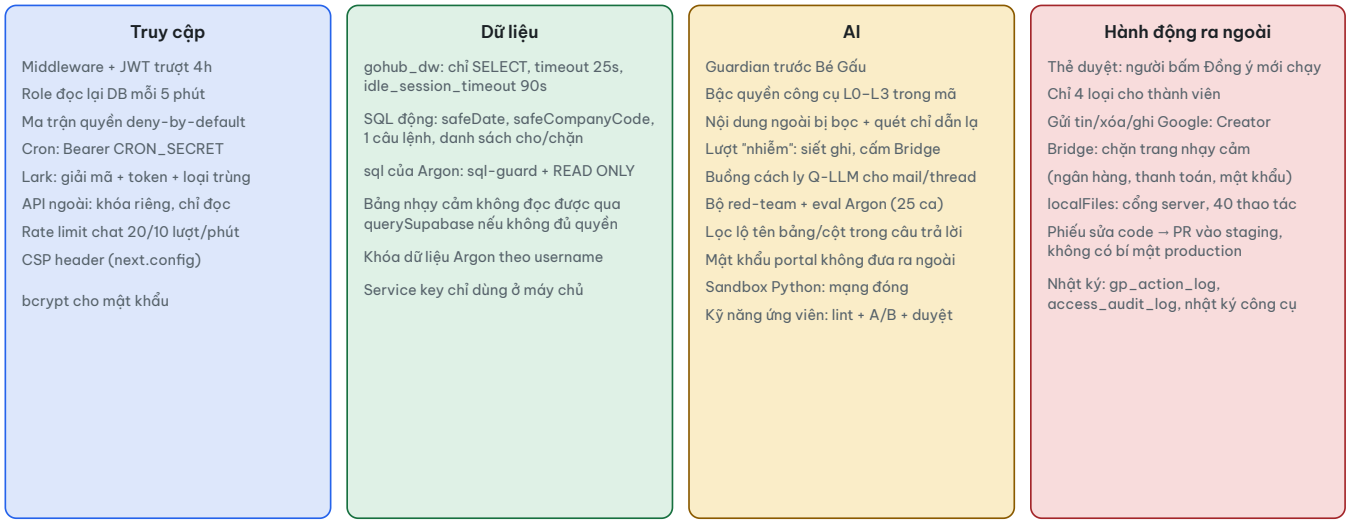
ROUTE	TẦN SUẤT	VIỆC
`scheduled-messages`	mỗi giờ (:01)	gửi tin Lark theo lịch, có cửa sổ bù 24 giờ
`etl-cache-sync`	mỗi 10–15 phút	đọc `jobs/job_logs` của gohub_dw; ETL mới xong thì soft-expire cache và làm nóng ~30 URL
`argon-sync`	hằng đêm	đồng bộ nguồn Lark/mail/thread, chỉ mục, sự việc, tuần tra đêm, xuất hồ sơ vendor (nếu bật)
`habit-learn`	hằng đêm	học thói quen ngầm của người dùng
`datapool-audit`	hằng đêm	soát giá vốn Datapool, DM Lark khi lệch cần xác nhận đã thêm



Cron-job.org và GitHub Actions không đảm bảo giờ chính xác; vì vậy các route gửi tin dùng cơ chế giành slot (optimistic lock) và cửa sổ bù.

13. Bảo mật và an toàn

Phòng thủ nhiều lớp. Với trợ lý AI, nguyên tắc chính là **an toàn nằm trong mã, không dựa vào lời dặn của prompt**.



Nguồn: `operations-runbook.md` §6, `run-safety.ts`, `bridge-safety.ts`, `access-pure.ts`.

Rủi ro và nợ bảo mật đã biết

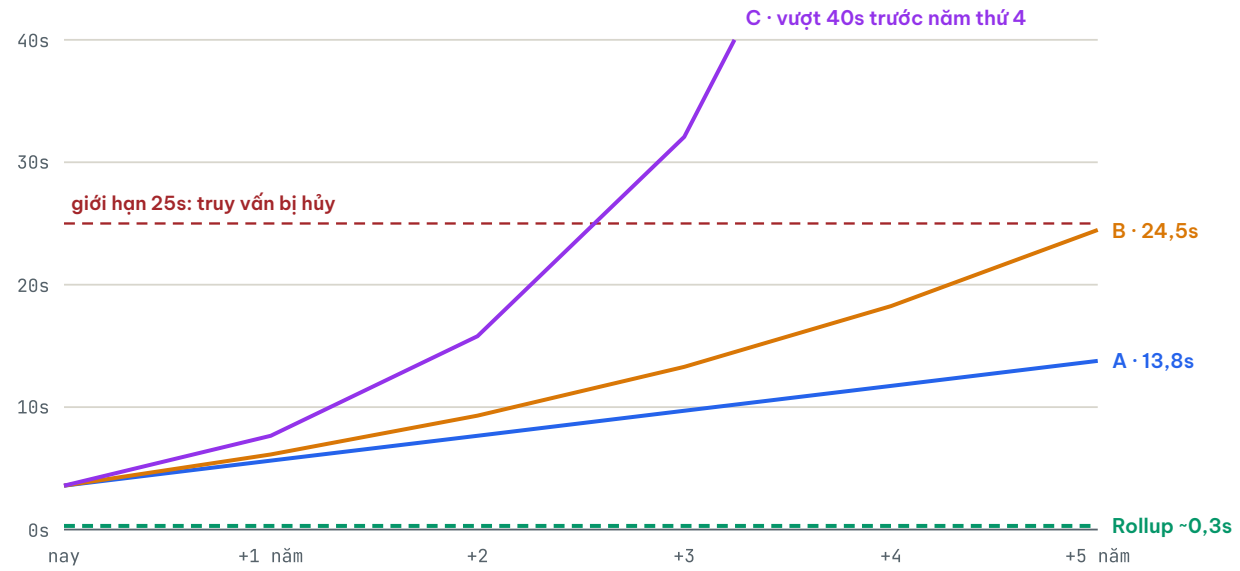
MỤC	TRẠNG THÁI	HÀNH ĐỘNG
Repo GitHub công khai	`lưu ý`	không commit bí mật; `.env*`, `CLAUDE.md`, `Hieu/` nằm ngoài git (kiểm trước mỗi commit)
Mật khẩu portal vendor trong `creator_kb` (chữ rõ)	`chưa xử lý`	chuyển sang trình quản lý mật khẩu rồi xoá khỏi KB; Argon đã không đưa ra ngoài
`VERCEL_TOKEN` cũ trong `web/.env.local`	`cần thu hồi`	bị Vercel từ chối; tạo token mới nếu cần
SSL tới gohub_dw	`một phần`	xác minh đầy đủ khi có `ANALYTICS_DB_SSL_CA`; DW không do GoHub sở hữu
Giới hạn kích thước body request	`chưa có`	rủi ro thấp (công cụ nội bộ)
Rate limit liên instance	`cần xác nhận`	Upstash đã có mã; kiểm biến môi trường trên Vercel
Staging dùng chung DB với production	`có chủ đích`	dữ liệu thử trên staging là dữ liệu thật; luôn dọn

14. Hiệu năng, giới hạn và kế hoạch Rollup

Phần này gộp số đo thật, dự đoán quy mô và hướng đã chốt. Chi tiết đầy đủ ở `docs/plans/analytics-rollup.md` và bản PDF `ANALYTICS-ROLLUP-ke-hoach.pdf`.

Số đo (28/10/2026) đã kiểm

PHÉP ĐO	KẾT QUẢ
Quét thuần `COUNT(*)` bảng fact giao hàng	≈ 0,35s lần đầu (đã trừ ~0,3s mạng)
Truy vấn kiểu BOD (nhóm kênh × tháng, 10 tháng, JOIN)	1,5–3,7s lần đầu; 2,3–5,7 µs mỗi dòng của cả bảng
Ép kiểu `::date` so với so chuỗi	786ms so với 611ms lần đầu: không phải thủ phạm chính
Cache nguội các route chính	bod-summary ~12s, staff ~6s, bod-b2b-tiers ~6s, quarterly-b2b-customers ~5s
Cache nóng	< 1s
Độ trễ kho	gohub_dw ~30ms/truy vấn đơn giản · Turso ~150ms · Supabase 280–450ms · Runtime Cache 4–15ms
Số dòng sau khi gộp (22 tháng)	ngày×kênh×công ty×khách×ship 205.376; ngày×kênh×công ty×SKU 258.353 (từ 652.741 dòng thô)



A: phẳng (~370k dòng/năm, đúng thực tế 2025–2026); B: +25%/năm; C: gấp đôi mỗi năm (kịch bản căng). Thời gian = số dòng × 5,5 µs (cận trên đo). Đường rollup là **ước tính**, sẽ đo thật ở giai đoạn P0.

Hướng đã chốt (2026-10-28)

Ràng buộc

- `gohub_dw` chỉ đọc, không sửa/thêm gì.
- Không phát sinh chi phí (Turso free, Vercel Hobby, cron-job.org).
- Quy mô tương lai suy ra từ dữ liệu bằng kịch bản.
- Số liệu các tab không đổi; chỉ bật sau khi đối chiếu khớp.

Thiết kế

- Turso `intel-rollup`: `ru_cust_day`, `ru_sku_day`, bản sao `dm_*`, `ru_meta`.
- Cron làm mới sau ETL (hằng giờ), chỉ ghi ngày có băm đổi; đối chiếu tháng hằng đêm.
- Router chọn bảng nhỏ nhất đủ trả lời; luôn có đường thô dự phòng.
- Thứ tự: Quarter Report → B2B → B2C → Channels → các tab khác.

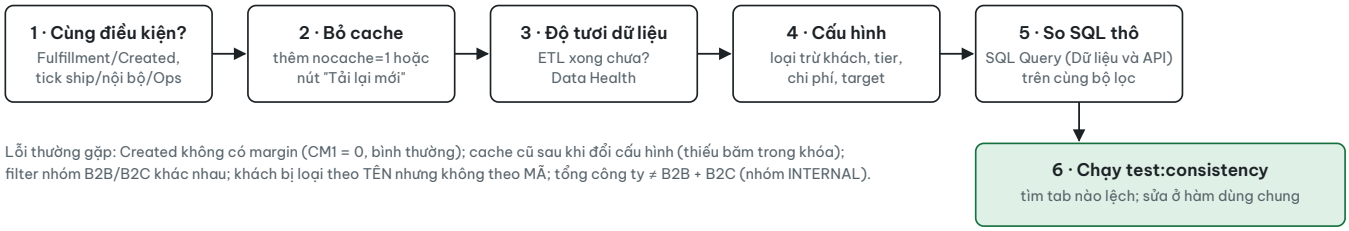
> Khi dữ liệu bị sửa (đổi nhãn, sửa giá trị fact, đổi mã): nhãn cập nhật sau lần làm mới kế (≤ 1 giờ); giá trị fact ≤ 1 giờ trong 45 ngày gần nhất, xa hơn ≤ 24 giờ; cache cần ba lớp xử lý (khóa theo `data_version`, bỏ cache biên cho route rollup, thêm version vào khóa stale-first). Điểm yếu duy nhất là số đo nướng sẵn theo nhóm vendor (ví dụ `hk3_rev_vnd`): băm bảng ánh xạ để dựng lại ngay khi đổi.

15. Vận hành và xử lý sự cố

Công cụ giám sát có sẵn

CÔNG CỤ	DÙNG ĐỂ	VỊ TRÍ
Giám sát dữ liệu (Data Health)	độ tươi (ETL đã chạy chưa), bất thường số liệu, đối chiếu giữa các tab	Dữ liệu và API › Giám sát
Data Map	bản đồ nguồn dữ liệu tự sinh, thời gian cache, nhãn "Dữ liệu cập nhật"	Dữ liệu và API
`perf-probe` , `perf_events`	đo độ trễ từng kho và thời gian từng route	API creator
Usage Analytics / My Metrics	mức dùng, chi phí AI, Tasks via Bé Gấu	Creator
`cron-alert`	cron lỗi thì DM Lark cho Creator	tự động
Chạy song song Gấu Pro / Gấu Prime	số lượt, chất lượng, tiêu chí gỡ Gấu Pro	Gấu Prime › Thêm
`test:consistency`	so số giữa các tab (chống lệch)	repo, chạy bằng `CONSISTENCY_BASE_URL`

Quy trình kiểm khi "số liệu sai hoặc lệch giữa hai tab"



Sửa ở hàm dùng chung (`lib/analytics-helpers.ts` , `lib/analytics-engine/*`) rồi chạy bộ kiểm nhất quán, đừng vá từng route.

Sự cố thường gặp

SỰ CỐ	DẤU HIỆU	XỬ LÝ
Cạn kết nối `gohub_dw`	"remaining connection slots...", nhiều route 500	pool đã giới hạn 3/instance + `idle_session_timeout` 90s; đợi nhả slot, tránh gọi song song nhiều route nặng; có retry 3 lần
Hết hạn mức Gemini	trợ lý báo lỗi hạn mức	kiểm quota; đổi model qua biến `GEMINI_MODEL*` (tập trung ở `lib/ai-models.ts`); hệ thống tự báo khi có model mới
Cron không chạy	không có tin/KPI buổi sáng	xem log Vercel + cron-job.org; kiểm `CRON_SECRET` ; `cron-alert` sẽ DM nếu route lỗi
Bot Lark không trả lời	không có phản hồi trong chat	kiểm bot còn trong group, `LARK_*` , loại trùng event, token hết hạn; xem `/api/lark/events`
Supabase gần đầy	nhắc dung lượng, ghi lỗi	không thêm dữ liệu lớn; xem bảng lớn (`listings` , `ncc_worldmove` , `skus`); dùng Turso cho dữ liệu mới
Staging làm bẩn dữ liệu thật	cuộc thử còn lại	xoá cuộc thử bằng API; nhớ DB dùng chung

Deploy lỗi	production hỏng sau merge	promote lại bản trước trên Vercel; sửa trên staging rồi merge lại (migration DROP chỉ chạy sau merge)

Chu kỳ kiểm

- **Hằng ngày (tự động):** ETL, cron buổi sáng, `cron-alert`, tin báo lỗi.
- **Hằng tuần:** xem Data Health (độ tươi/bất thường/đối chiếu), chi phí AI, dung lượng Supabase, lỗi trong log Vercel.
- **Hằng tháng:** xoay bí mật cần thiết, rà quyền người dùng, chạy bộ eval trợ lý, kiểm hạn mức Turso (đặc biệt khi bắt đầu Rollup).

Phụ lục A · Bảng dữ liệu theo miền

Supabase (Postgres)

MIỀN	BẢNG CHÍNH
Catalog và NCC	`products`, `skus`, `listings`, `items`, `ncc_worldmove`, `ncc_3hk`, `ncc_datapool`, `vendor_quotes`, `ref_vendors`, `ref_countries`, `ref_support_countries`, `sku_vat`, `pm_operators`, `pm_price_lists`, `bc_*`
Tri thức và wiki	`kb_wiki_pages`, `kb_wiki_page_groups`, `kb_chunks`, `creator_kb`
Người dùng và cấu hình	`users`, `app_settings` (cấu hình, quyền, cờ, trạng thái phụ), `external_api_keys`, `access_audit_log`
Trợ lý chung	`conversation_messages`, `app_usage_events`, `chat_feedback`, `chat_questions`, `chatbot_learning_log`, `assistant_memory`, `lark_chat_history`
Gấu Pro	`gp_pending_actions`, `gp_runs`, `gp_jobs`, `gp_conversation_memory`, `gp_scheduled_tasks`, `gp_action_log`, `dev_tickets`, `trend_snapshots`
Gấu Prime (Argon)	`argon_projects`, `argon_missions`, `argon_runs`, `argon_sources`, `argon_source_versions`, `argon_folders`, `argon_facts`, `argon_mail_threads`, `argon_workspace_files`, `argon_mission_sources`
Bridge	`browser_bridge_commands`, `browser_bridge_devices`, `browser_bridge_pairings`
My Metrics (OKR)	`okr_evidence_records`, `okr_lark_events`, `okr_lark_message_log`, `okr_plan_items`, `okr_sku_tags`
Analytics phụ trợ	`analytics_monthly_kpis`, `b2b_customer_targets`, `b2b_customer_cost_monthly`, `company_monthly_targets`, `squad_targets`, `inventory_plan_*`, `inventory_po`, `ca_thread_log`

Turso

DATABASE	BẢNG
Cấu hình (`TURSO_URL`)	`b2b_customer_cost_monthly`, `b2b_ecom_cost_monthly`, `target_planning_quarter`, `cs_tickets`, `manual_edit_history`, `log_archive`, `perf_events`, `url_hits`, `country_codes`
argon-hub (`ARGON_TURSO_URL`)	`argon_chunks`, `argon_index`, `argon_entities`, `argon_edges`, `argon_mentions`, `agent_events`
Leads (`TURSO_LEADS_URL`)	dữ liệu Omni (khách tiềm năng)
Sắp có (kế hoạch)	`intel-rollup`: `ru_cust_day`, `ru_sku_day`, `dm_*`, `ru_meta`, `ru_runlog`

Biến môi trường theo nhóm (chỉ tên, không có giá trị)

NHÓM	BIẾN
Kho số liệu	`ANALYTICS_DB_HOST/PORT/NAME/USER/PASSWORD/SSL_CA`
Supabase	`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_KEY`
Turso	`TURSO_URL/AUTH_TOKEN`, `ARGON_TURSO_URL/AUTH_TOKEN`, `TURSO_LEADS_URL/AUTH_TOKEN`

Xác thực	`NEXTAUTH_SECRET`, `NEXTAUTH_URL`, `CRON_SECRET`, `MCP_SECRET`, `DEV_TICKET_SECRET`, `PORTAL_CRED_KEY`
Lark	`LARK_APP_ID/SECRET`, `LARK_ENCRYPT_KEY`, `LARK_VERIFICATION_TOKEN`, `LARK_BOT_NAME`, `LARK_CREATOR_USER_ID`, `LARK_BASE_ID/TABLE_ID`, `LARK_OAUTH_SCOPES`
Gemini	`GEMINI_KEY`, `GEMINI_MODEL`, `GEMINI_MODEL_PRO`, `GEMINI_MODEL_GAU_PRO`, `GEMINI_MODEL_ARGON`, `GEMINI_LIVE_MODEL`, `GEMINI_TTS_MODEL`, `GEMINI_IMAGE_MODEL(_PRO)`, `GEMINI_VIDEO_MODEL(_HQ)`, `GEMINI_TRANSLATE_MODEL`, `GEMINI_TRANSCRIBE_MODEL`, `GEMINI_DEEP_RESEARCH_AGENT`
Dịch vụ khác	`GOOGLE_OAUTH_CLIENT_ID/SECRET`, `BROWSERLESS_WS_URL/TOKEN`, `ARGON_SANDBOX_TOKEN`, `UPSTASH_REDIS_REST_URL/TOKEN`, `CHATWOOT_*`, `OMNI_API_*`, `BC_DATAPOOL_*`, `ADMIN_GOHUB_*`, `VERCEL_TOKEN/TEAM_ID/PROJECT_ID`, `GITHUB_DISPATCH_TOKEN`

Phụ lục B · Thuật ngữ

THUẬT NGỮ	NGHĨA
Fulfillment / Created	cơ sở thời gian báo cáo: ngày giao hàng / ngày tạo đơn
GP, GPM %, CM1, CM1 %	Gross Profit, tỷ suất GP, Contribution Margin 1 (GP trừ chi phí vận hành), tỷ lệ CM1
Strategic / VIP / Gold / Silver	phân hạng khách B2B theo price list
Datapool	sản phẩm dung lượng dùng chung của BC Datapool và 3HK Datapool
NCC	nhà cung cấp; "Gap" = gói NCC chưa có SKU
Sứ mệnh (Mission)	không gian làm việc của Gấu Prime: brief, kế hoạch, workspace, lượt hỏi; mỗi cuộc trò chuyện là một sứ mệnh
Kho	nơi chứa tài liệu, ghi chú, mail, thread, sự việc, thực thể của Gấu Prime
Sự việc (fact)	thông tin có hạn trích từ nguồn: hạn, khuyến mãi, ngừng bán, đổi giá
Bậc quyền L0–L3	mức rủi ro của công cụ Argon (đọc tin cậy / đọc ngoài / ghi hoàn tác / cần người quyết)
"Nhiễm" (tainted)	lượt đã đọc nội dung ngoài: bị siết ghi và cấm điều khiển trình duyệt
Bridge	extension trình duyệt cho trợ lý thao tác trên máy người dùng
SWR	stale-while-revalidate: trả bản cũ rồi làm mới ở nền
Rollup	bảng tổng hợp sẵn theo ngày, thay vì tính từ bảng thô mỗi lần xem

Phụ lục C · Việc còn mở, nợ kỹ thuật, điều chưa biết

- **Rollup Analytics:** toàn bộ P0–P6 chưa làm; cần audit từng route trước khi code (chương 14).
- **Gấu Prime:** chưa kiểm sống bằng tài khoản thành viên thật (run_code, xuất file, Kho báo cáo, đính kèm); eval còn 2 ca bị trừ điểm vì bịa chi tiết; nút giao diện cho "nhờ Argon sửa tài liệu" của thành viên chưa có; PDF một chạm chưa có.
- **Gấu Pro:** còn giữ vì Gấu Prime chưa chuyển hết: Bridge điều khiển máy, giọng nói Live, panel Trí nhớ hợp nhất, tìm hội thoại theo ý nghĩa, verifyReportNumbers.
- **Dữ liệu:** một số mã thừa đọc `partner-tiers` (đang rỗng) ở trang B2B/Customers/Dashboard; định nghĩa "doanh thu" mặc định của bot cần chốt; "KH cũ chưa quay lại" ở tab Quarter đếm cả lịch sử.
- **Hạ tầng:** Upstash chưa chắc đã cấu hình; cron-job.org còn việc cấu hình thủ công; `VERCEL_TOKEN` cũ cần thu hồi; mật khẩu portal trong KB cần chuyển đi.

- **Chưa biết (sẽ đo):** lịch và thời lượng ETL đơn tạo; ETL có sửa số cũ không; độ trễ đọc rollup trên Turso; kiểu và độ nhất quán các cột ngày ở các bảng ngoài fact giao hàng.

Phụ lục D · Nguồn tài liệu

- `CLAUDE.md` (trạng thái hiện tại, quy tắc, checklist), `docs/session_summary.txt` (nhật ký từng phiên).
- `docs/wiki/system/kien-truc-he-thong.md`, `operations-runbook.md`, `analytics-data-model.md`, `chatbot-agents-guardian.md`, `quy-trinh-van-hanh.md`.
- `docs/wiki/system/tabs/*.md` (một file mỗi tab, gồm phần "Gotchas"), `docs/wiki/business/*.md` (nghiệp vụ sản phẩm, vendor, giá).
- `docs/plans/analytics-rollup.md` và `ANALYTICS-ROLLUP-ke-hoach.pdf`.
- Mã nguồn: `web/src/lib/analytics-helpers.ts`, `analytics-engine/`, `agents/`, `argon/`, `nav.ts`; `web/vercel.json`; `.github/workflows/`.

Tài liệu này là ảnh chụp tại 2026-10-28. Khi kiến trúc đổi (đặc biệt sau Rollup), cập nhật cả nguồn HTML `docs/system-doc/he-thong.html` rồi dựng lại PDF.